Reinforcement Learning

CSC 480: Artificial Intelligence, Spring 2026
Professor Austin P. Wright: Course Notes

Abstract
We will cover a range of the core methods of reinforcement learning: offline model based reinforcement learning, model-free learning, online learning and the exploration/exploitation trade-off, q-learning, and approximate reinforcement learning.

Contents

I Reinforcement Learning

As we have found, Markov Decision Processes provide the mathematical language to describe the behaviour of rational agents acting in an uncertain world. However, we have failed to take into account the *meta-uncertainty* about the structure of the environment that we most frequently face in practice. That is to say, what do we do if we still have an MDP with:

- States: S
- Actions: A
- Transition Model: $T(s, a, s')$
- Reward Function: $R(s, a, s')$

And we want to find an optimal policy $\pi(s)$, but *we do not know T or R*. Instead all we have is what the environment tells us, and so while we still need to maximize expected reward, all learning must be based on observed samples of outcomes.

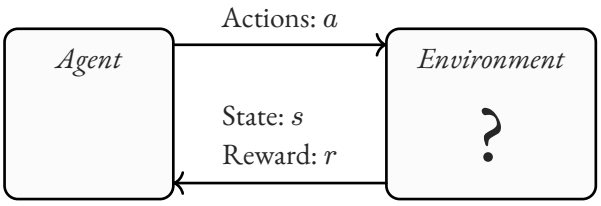


Figure 1: Diagram of Classical Reinforcement Learning

Offline vs Online Learning

Our previous approach to solving MDPs have been *offline*. What this means is that the agent is able to effectively reason about what it expects to happen *beforehand*, and then choose the best option right away. In offline learning an agent already knows not to touch a hot stove. In *online learning* the agent only knows what it can experience, it must actually touch the stove in order to learn that hard way that it is too hot.

What remains for online reinforcement learning, and the focus of the rest of the notes, are the following components of the problem:

1. How do we learn from the data collected by interacting with the environment
2. How do we act when we dont know much about the environment
3. How can we approximate these procedures in environments where complete models of the states are not possible

II Model-based RL

For this part we want to figure out how to learn the relevant components of the MDP from data of actions in the world. In order to do this let us start with the assumption of some history of "episodes" of training data which is a set of traces of samples of what happens by taking an action in a given state:

$$\begin{aligned} \text{sample} &= (s_i, a_i, s_{i+1}, r_i) \\ \text{episode} &= [(s_0, a_0, s_1, r_0), ..., (s_i, a_i, s_{i+1}, r_i), ...] \\ \text{data} &= \{\text{episode } 1, \text{episode } 2 \ ...\} \end{aligned} \tag{1}$$

The model-based approach to reinforcement learning is the most conceptually simple, and in many cases is the best choice[1]. All that the agent needs to do is take the sampled training data, and use it to generate an approximate transition function $\hat{T}(s, a, s')$[2]. Thus the method is simply described as:

1. Keep track of the counts of each observed transition (s, a, s', r).
2. When calculating an action, normalize the samples so that the transition function is a valid probability distribution.
3. Solve the resulting MDP as normal using value or policy iteration.

Discussion

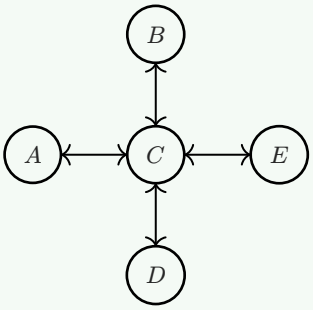
Above I have broken down the sample data by episode (that is by the separate instances of traces of continuous actions and consequences). However, is this always required when using the data to estimate the transitions, or can all of the samples be lumped together? Why or why not? Consider the Markov Property. Additionally, are there other situations where breaking things down by episode is required?

[1] Just because it will take less time to cover does not mean it is any less important, there is lots of ongoing active research in model-based approaches

[2] Here using the common notation of a $\hat{}$ over a sample based empirical reconstruction of some uncertain quantity.

Discussion

Consider the following MDP with states A, B, C, D, E, and terminal T.



Now consider the following data:

Episode 1:
- B, down, C, -1
- C, down, D, -1
- D, exit, T, $+10$

Episode 2:
- B, down, C, -1
- C, down, D, -1
- D, exit, T, $+10$

Episode 3:
- E, left, C, -1
- C, down, D, -1
- D, exit, T, $+10$

Episode 4:
- E, left, C, -1
- C, down, A, -1
- A, exit, T, -10

Based on this sample data, calculate:
1. The estimated transition function
2. The estimated reward function
3. The optimal policy[3]

[3] Notice how the discount γ never appears in the sample data. It is purely a function of the optimization calculation and as such is accounted for *after* reconstructing the MDP model.

III Model-free RL

While the model-based approach gives the benefits of monotonically increasing knowledge of the environment, and thus clear progress towards optimality, sometimes it is not possible to store all of the data involved in the full transition function. In such a situation we can notice that if samples are really representative, we can skip the distribution all together. If the value of an action is the expected value of the results of that action, consider the three scenarios for calculating an expected value:

1. Known probability: $E[X] = \sum_x P(x) \cdot x$
2. Model probability: $\hat{P}(x) = \frac{\text{count}(x)}{N}$ $\quad E[X] \approx \sum_x \hat{P}(x) \cdot x$
3. Model-free: $E[X] \approx \frac{1}{N} \sum_i x_i$

We can see how if samples appear with the correct frequency, their naive average will converge to the expected value without explicit calculation of the probability itself. This is the core idea of model-free learning.

Direct Evaluation

The most simple model-free approach is direct evaluation. In direct evaluation we just set some fixed policy π, and have the agent run through many episodes using that policy. Then for each visited state, every time you visit mark the sum of what the discounted rewards turned out to be. Finally average everything out and you have an estimate of the values under that policy: $\hat{V}^\pi$. This approach, however, has some serious problems. It does not take advantage of the connections between states and so each state must be learned separately and thus it takes a long time. What we want is a way to integrate something like the Bellman equation into this process.

> ***Discussion***
> Return to the previous example problem for model based learning. Notice how all of the actions in the samples follow a single policy. Calculate the value of that policy based on the provided samples using Direct Evaluation.

> ***Discussion***
> In direct evaluation model-free learning, unlike model-based learning, why does breaking down samples into episodes matter?

Temporal-Difference Learning

Temporal Difference Learning (TDL) provides the better way to estimate the values of states for a fixed policy. The central idea is to update the value $V(s)$ every single time we experience a transition (s, a, s', r), taking into account the Bellman equation of the value combining the transition reward with the value of the outcome.

We first calculate the implied value of a state from the sample (s, a, s', r):

$$V^{\pi}_{\text{sample}}(s) = r + \gamma \hat{V}^{\pi}(s') \tag{2}$$

We then keep track of the values of the states we see with an exponential moving average (with parameter α which is called the *learning rate*) updating the values with each sample:

$$\hat{V}^{\pi}(s) \leftarrow (1-\alpha)\hat{V}^{\pi}(s) + \alpha V^{\pi}_{\text{sample}}(s) \tag{3}$$

Or viewed as an iterative update procedure:

$$\hat{V}^{\pi}_{k+1}(s) = (1-\alpha)\hat{V}^{\pi}_{k}(s) + \alpha\left[r + \gamma \hat{V}^{\pi}_{k}(s')\right] \tag{4}$$

> ***Discussion***
> Consider the parameter α. What is it doing in this equation? What would the method reduce to if we set it to the minimum value of 0? What about the maximum value of 1? When would we prefer higher or lower values?

Q-Learning

While TDL is an efficient, model-free, method to evaluate a specific policy, if we want to turn these values into an *optimal policy* we have a major issue. If the TDL values are policy dependent, we would need to recalculate the whole TDL proceedure for each change of the policy to very very slowly improve (with no assurance of finding the optimal policy). Instead we want to find a way to integrate the policy optimization into the whole learning process.

Recall that instead of state values sometimes we prefer to model state-action values, or Q-values. Can we do TDL but for Q-values? The answer is yes, and the method is called Q-learning.

Q-learning is essentially the same as TDL, but instead of using the sample based aggregation to look over q nodes to successor states, we can use the same sample based aggregation to look over state nodes to successor q values. We update Q values using the same pseudo-Bellman and moving average approach as in TDL. So for each sample transition (s, a, s', r) we perform an update:

$$Q(s,a) \leftarrow (1-\alpha) \cdot Q(s,a) + \alpha \cdot \left[r + \gamma \cdot \max_{a'} Q(s',a') \right] \tag{5}$$

Notice how we no longer have to rely on a fixed policy. Our policy consists just of maximizing Q values: $\pi(s) = \max_a Q(s,a)$.

Amazingly, Q-learning will eventually converge to the optimal policy if ran for long enough (even if earlier policies are suboptimal). This process of learning being relevant between different policies is called *off-policy learning*.

The caveats are that you have to explore the whole space enough, and eventually lower the learning rate enough[4]. But in the limit, it does not matter how you select actions and eventually you will be able to converge to an optimal policy.

[4] But not too quickly or else you wont explore or learn in the early stages!

Discussion

Imagine an unknown environments with three states A, B, C, terminal state T, and two actions $\leftarrow$ and $\rightarrow$. An agent acting in this environment has recorded the following episode repeated infinitely:

s	a	s'	r	Iteration Number
A	$\rightarrow$	B	0	$1, 10, 19, \ldots$
B	$\rightarrow$	C	0	$2, 11, 20, \ldots$
C	$\leftarrow$	B	0	$3, 12, 21, \ldots$
B	$\leftarrow$	A	0	$4, 13, 22, \ldots$
A	$\rightarrow$	B	0	$5, 14, 23, \ldots$
B	$\leftarrow$	A	0	$6, 15, 24, \ldots$
A	$\rightarrow$	B	0	$7, 16, 25, \ldots$
B	$\rightarrow$	C	0	$8, 17, 26, \ldots$
C	$\leftarrow$	T	1	$9, 18, 27, \ldots$

- Based on just these samples, what do you think the MDP would look like? Draw the network, then using a model-based approach calculate the transition functions of the MDP.
- Now consider an agent doing Q-learning seeing the sequence of samples above. After which iteration (if any) do the following quantities first become nonzero?
  - $Q(A, \rightarrow)$:
  - $Q(B, \rightarrow)$:
  - $Q(B, \leftarrow)$:
  - $Q(C, \rightarrow)$:
  - $Q(C, \leftarrow)$:

IV Choosing Actions

Exploration vs Exploitation

ε-Greedy Actions

V Approximate RL